NLEAD RAG Notebook Guide

Documentation for NLEAD_RAG.ipynb - how each step works, what the code means, and what outputs to expect. This notebook pulls SharePoint lists and document libraries through Microsoft Graph, saves JSON for RAG, and pre-chunks text.

Pipeline overview:

- Load Python libraries and .env credentials
- Configure Azure / SharePoint settings
- Authenticate to Microsoft Graph (app-only or device-code)
- Resolve the SharePoint site ID
- Export lists to sharepoint_lists.json
- Export document libraries to sharepoint_libraries.json
- Merge into sharepoint_combined.json for RAG
- Pre-chunk document text for embedding

Step 1 - Imports and environment loading

Purpose: import libraries used across the notebook and load secrets from a .env file so credentials are not hard-coded in cells.

Key imports

- msal - Microsoft Authentication Library (tokens for Graph)
- requests - HTTP calls to Microsoft Graph API
- pandas - preview exported data as tables
- dotenv.load_dotenv - read AZURE_* and SHAREPOINT_* from .env
- json / pathlib / datetime - file I/O and timestamps

```
from dotenv import load_dotenv
load_dotenv(override=True)  # reload .env on every run
```

Note: override=True means values in .env replace any already set in the process environment.

Step 2 - Configuration

Purpose: define tenant/app credentials, target SharePoint site, auth mode, Graph scopes, and the output folder for JSON files.

Important variables

- TENANT_ID / CLIENT_ID / CLIENT_SECRET - Azure app registration identity
- SHAREPOINT_SITE_URL - site root (e.g. https://amenhanna.sharepoint.com)
- LIST_NAMES / LIBRARY_NAMES - empty lists mean export all (non-hidden) lists/libraries
- AUTH_MODE - auto | client_credentials | device_code | auth_code
- GRAPH_BASE - https://graph.microsoft.com/v1.0
- SCOPES - app-only scope (.default); DELEGATED_SCOPES - Sites.Read.All, Files.Read.All
- JSON_OUTPUT_DIR - data/sharepoint_json/

Auth modes

- auto - try app-only first; if required application permissions are missing, use device-code
- client_credentials - app-only (needs Application permissions + admin consent)
- device_code - interactive sign-in in a browser with a short code (best for notebooks)
- auth_code - browser redirect to localhost (needs Web redirect URI in Azure)

Step 3 - Authenticate with Microsoft Graph

Purpose: obtain an ACCESS_TOKEN and build HEADERS used by every Graph request.

Helper functions

- decode_token_claims / roles / scopes - inspect the JWT to see granted roles or scp scopes
- _acquire_client_credentials_token - confidential client; no user login
- _acquire_device_code_token - prints a code; user signs in at microsoft.com/devicelogin
- _acquire_auth_code_token - opens browser and waits for localhost redirect
- get_access_token - picks the mode, validates permissions, returns (token, mode)

```
ACCESS_TOKEN, ACTUAL_AUTH_MODE = get_access_token()
HEADERS = {"Authorization": f"Bearer {ACCESS_TOKEN}"}
```

In auto mode, if the app-only token is missing Application roles Sites.Read.All and Files.Read.All, the notebook falls back to device-code. For device-code to work, enable 'Allow public client flows' on the Azure app and grant Delegated Sites.Read.All + Files.Read.All.

Note: Long-term fix for unattended runs: add Application permissions Sites.Read.All and Files.Read.All, then Grant admin consent. App-only auth will then succeed without a browser.

Step 4 - Graph helpers and site lookup

Purpose: reusable GET helper with pagination, plus resolve the SharePoint site to a Graph site ID.

Functions

- graph_get(url) - GET with Bearer token; follows @odata.nextLink until all pages are collected
- parse_sharepoint_site_url - splits URL into hostname and path
- build_site_graph_ref - builds Graph site key (root site uses hostname:/)
- get_site_id - GET /sites/{hostname}:/path and returns site['id']

```
SITE_ID = get_site_id(SHAREPOINT_SITE_URL)
# Example output:
# Site: RAG AI Agentic Deployment Site (Test) (amenhanna.sharepoint.com,...)
```

SITE_ID is required for later calls that list site lists and drives.

Step 5 - Export SharePoint lists to JSON

Purpose: download list definitions and items (with fields expanded) into sharepoint_lists.json.

Functions

- get_sharepoint_lists(site_id) - GET /sites/{id}/lists
- get_list_items(site_id, list_id) - GET items?expand=fields; keeps id, urls, dates, fields
- export_lists_to_json - filters by LIST_NAMES or skips hidden/system lists; writes JSON

Output shape (sharepoint_lists.json)

```
{
  "source": "sharepoint_list",
  "siteUrl": "...",
  "exportedAt": "...",
  "lists": [ { "displayName", "itemCount", "items": [ ... ] } ]
}
```

Example from a successful run: After_Action_Reports (637), Survey_Data (256), Course_Curriculum (40), Registration_Data (256), and others - saved under data/sharepoint_json/sharepoint_lists.json.

Step 6 - Preview lists in a DataFrame

Purpose: flatten list items into rows for a quick pandas preview. Each row includes the list name plus all field columns from the item.

```
for lst in lists_data['lists']:
    for item in lst['items']:
        row = {'list': lst['displayName'], **item['fields']}
        rows.append(row)
lists_df = pd.DataFrame(rows)
```

Step 7 - Export document libraries to JSON

Purpose: walk each drive (document library), collect files recursively, optionally download text content for supported extensions, and write sharepoint_libraries.json.

Functions

- get_document_libraries - GET /sites/{id}/drives
- list_drive_items_recursive - children of root, then recurse into folders
- download_file_text - GET /drives/{id}/items/{id}/content for text-like files
- export_libraries_to_json - builds library/file records; TEXT_EXTENSIONS get content

TEXT_EXTENSIONS currently: .txt, .md, .csv, .json, .html, .htm. Binary files such as docx, pdf, and pptx are listed with metadata only; their content field stays null. A placeholder string is used later in the combined document.

Note: Office/PDF extraction is not implemented yet. For RAG over those files, add parsers (e.g. python-docx, pypdf) in a later step.

Step 8 - Preview library metadata

Purpose: show a compact table of library, file name, size, whether text content was downloaded, and webUrl.

Step 9 - Merge into one JSON for chunking

Purpose: normalize lists and libraries into a single documents[] array that the chunking / embedding pipeline can consume.

Document schema

- sourceType - sharepoint_list or sharepoint_document_library
- sourceName - list or library display name
- id / webUrl - Graph identifiers and SharePoint links
- text - JSON-serialized list fields, or file text / binary placeholder

```
combined = {
  "source": "sharepoint_combined",
  "documents": list_items_to_documents(...) + library_files_to_documents(...)
}
# Saved as data/sharepoint_json/sharepoint_combined.json
```

Example run produced 1223 combined documents ready for pre-chunking.

Step 10 - Pre-chunking

Purpose: split each document's text into overlapping windows for embedding/retrieval. Defaults: CHUNK_SIZE=1000 characters, CHUNK_OVERLAP=200.

```
def chunk_text(text, chunk_size, overlap):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size - overlap
    return chunks
```

Output chunks_df columns: sourceType, sourceName, webUrl, chunk_id, text. Next RAG steps (not in this notebook yet) typically embed chunks and store them in a vector index.

Appendix A - Azure Portal checklist

For app-only (client_credentials / preferred production path)

- App registration -> Certificates & secrets -> create a client secret
- API permissions -> Microsoft Graph -> Application -> Sites.Read.All, Files.Read.All
- Click Grant admin consent for the tenant
- Put AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET in .env
- Set GRAPH_AUTH_MODE=auto (or client_credentials)

For device-code (interactive notebook login)

- Authentication -> Advanced settings -> Allow public client flows = Yes
- API permissions -> Delegated -> Sites.Read.All, Files.Read.All (+ admin consent if required)
- Run the auth cell; open the printed URL and enter the device code

For auth_code (browser + localhost redirect)

- Authentication -> Web -> Redirect URI: http://localhost:8400 (or AZURE_REDIRECT_URI)
- Client secret required; GRAPH_AUTH_MODE=auth_code

Appendix B - Output files

- data/sharepoint_json/sharepoint_lists.json - lists + items + fields
- data/sharepoint_json/sharepoint_libraries.json - drives + file metadata (+ text content)
- data/sharepoint_json/sharepoint_combined.json - unified documents[] for RAG

Appendix C - Suggested run order

- 1. Ensure .env exists (copy from .env.example) with Azure + SharePoint values
- 2. Run import cell
- 3. Run configuration cell - confirm Tenant / Client / Auth mode / Site printout
- 4. Run authentication cell - complete device login if prompted
- 5. Run Graph helpers / site ID cell
- 6. Run list export + preview
- 7. Run library export + preview
- 8. Run combine cell
- 9. Run pre-chunking cell

Appendix D - Common errors

- App-only token missing permissions - grant Application Sites.Read.All + Files.Read.All
- Browser login timed out - redirect URI missing, or use device_code instead
- device flow / AADSTS7000218 - enable Allow public client flows
- Graph 401 Unauthorized - token lacks Sites/Files read rights or consent not granted